

Strings

Section 2

Chapter 2

An Introduction to Programming Using Python
David Schneider

© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.

Strings

- Topics
 - String literals and variables
 - Indices and slices
 - Concatenation and repetition
 - String functions and methods

String Literals

- Sequence of characters that is treated as a single item
- Written as a sequence of characters surrounded by either single quotes (') or double quotes (").

"John Doe"
'5th Avenue'
'76'
"Say it ain't so, Joe!"

Opening and closing
quotation marks must
be the same type

String Variables

- Variables also can be assigned string values
 - Example: `x = "Hello"`, `name1 = "John"`
- Created (come into existence) the first time they appear in assignment statements
- When an argument of a print statement
 - Quotation marks not included in display

Indices and Slices

- Position or index of a character in a string
 - Identified with one of the numbers 0, 1, 2, 3,

s	p	a	m		&		e	g	g	s
↑	↑	↑	↑	↑	↑	↑	↑	↑	↑	↑
0	1	2	3	4	5	6	7	8	9	10

FIGURE 2.4 Indices of the characters of the string "spam & eggs".

Indices and Slices

- If `str1` is a string, then `str1[m:n]` is the substring beginning at position `m` and ending at position `n - 1`
 - Example `"spam & eggs"[2:7]`

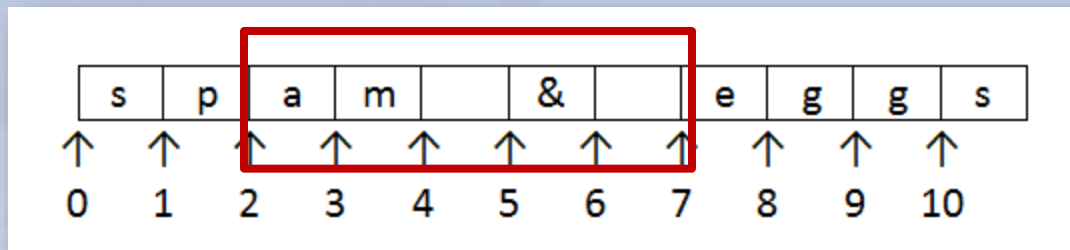


FIGURE 2.5 Aid to visualizing slices.

Indices and Slices

- Example 1: Program shows use of indices

```
print("Python")
print("Python"[1], "Python"[5], "Python"[2:4])
str1 = "Hello World!"
print(str1.find('W'))
print(str1.find('x'))
print(str1.rfind('l'))
```

[Run]

```
Python
y n th
6
-1
9
```


Negative Indices

- Python allows strings to be indexed by their position with regards to the right
 - Use negative numbers for indices.

s	p	a	m		&		e	g	g	s
↑	↑	↑	↑	↑	↑	↑	↑	↑	↑	↑
-11	-10	-9	-8	-7	-6	-5	-4	-3	-2	-1

FIGURE 2.6 Negative indices of the characters of the string "spam & eggs".

Negative Indices

- Example 2: Program illustrates negative indices.

```
print("Python")
print("Python"[-1], "Python"[-4], "Python"[-5:-2])
str1 = "spam & eggs"
print(str1[-2])
print(str1[-8:-3])
print(str1[0:-1])
```

[Run]

```
Python
n t yth
g
m & e
spam & egg
```

Default Bounds for Slices

- Example 3: Program illustrates default bounds

```
print("Python"[2:], "Python"[:4], "Python"[:])  
print("Python"[-3:], "Python"[:-3])
```

[Run]

```
thon Pyth Python  
hon Pyt
```

Your Turn ...

- Use `index([])` and 'slice'

- `S1 = "A red car "`

S1

[0] [1] [2] [3] [4] [5] [6] [7] [8] [9]

- `>>> S1[4]`

- 'd'

- `>>> S1[2:7]`

- 'red c'

- `>>> S1[:7]`

- 'A red c'

- `>>> S1[2:-6]`

- 're'

- `>>> S1[1] = '*'`

- `TypeError: 'str' object does not support item assignment`

- `>>> S1 = 'something else'`

- `# This is ok`

A		r	e	d		c	a	r	
---	--	---	---	---	--	---	---	---	--

[-10] [-9] [-8] [-7] [-6] [-5] [-4] [-3] [-2] [-1]

String Concatenation

- Two strings can be combined to form a new string
 - Consisting of the strings joined together
 - Represented by a plus sign
- Combination of strings, plus signs, functions, and methods can be evaluated
 - Called a string expression

String Repetition

- Asterisk operator can be used with strings to repeatedly concatenate a string with itself

Expression	Value
"ha" * 4	"hahahaha"
"mur" * 2	"murmur"
'x' * 10	"xxxxxxxxxx"
("cha-" * 2) + "cha"	"cha-cha-cha"

Your Turn ...

S1 [0] [1] [2] [3] [4] [5]

G	r	e	e	n	
----------	----------	----------	----------	----------	--

 [-6] [-5] [-4] [-3] [-2] [-1]

S2 [0] [1] [2] [3] [4] [5]

Y	e	l	l	o	w
----------	----------	----------	----------	----------	----------

 [-6] [-5] [-4] [-3] [-2] [-1]

- Concatenation: Combine 2 strings to form a new one
- S1 = “Green ” S1 = “Yellow”
- >>> **S1+S2**
 - ‘Green Yellow’
- >>> **S2+S1**
 - ‘YellowGreen ’
- Repetition: Repeatedly concatenate a string with itself
- >>> **S2*3**
 - ‘YellowYellowYellow’

String Functions and Methods

Function or Method	Example	Value	Description
len	len(str1)	6	number of characters in the string
upper	str1.upper()	"PYTHON"	uppercases every alphabetical character
lower	str1.lower()	"python"	lowercases every alphabetical character
count	str1.count('th')	1	number of non-overlapping occurrences of the substring
capitalize	"coDE".capitalize()	"Code"	capitalizes the first letter of the string and lowercases the rest
title	"beN hur".title()	"Ben Hur"	capitalizes the first letter of each word in the string and lowercases the rest
rstrip	"ab ".rstrip()	"ab"	removes spaces from the right side of the string

Table 2.3 String Operations (str1 = "Python")

Your Turn ...

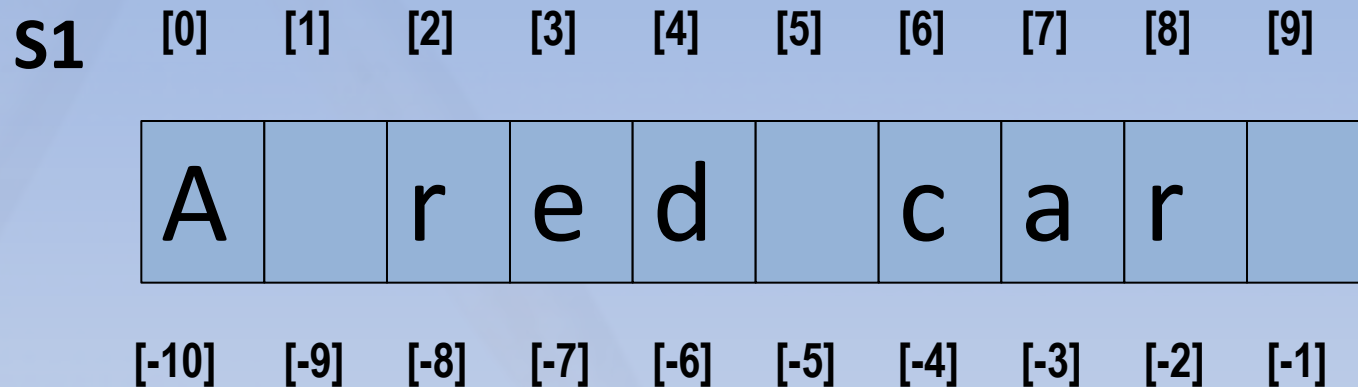
s1	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]
	A		r	e	d		c	a	r	
	[-10]	[-9]	[-8]	[-7]	[-6]	[-5]	[-4]	[-3]	[-2]	[-1]

- `S1.count('r') → 2`
- `S1.split() → ['A', 'red', 'car']`
- `S1.replace('r', '*') → "A *ed ca* "`
- `S1.index('r') → 2`
- `S1.index('x') → ValueError`

NOTE: This is not an exhaustive list

NOTE: This is not an exhaustive list

Your Turn ...



- S1.find('r') → 2;
- S1.find('x') → -1
- S1.startswith('A red') → True
- S1.upper() → "A RED CAR "

NOTE: This is not an exhaustive list

NOTE: This is not an exhaustive list

Chained Methods

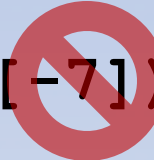
- Lines can be combined into a single line said to *chain* the two methods
 - Executed from left to right

```
praise = "Good Doggie".upper()  
numberOfGees = praise.count('G')
```



```
numberOfGees = "Good Doggie".upper().count('G')
```

Indexing and Slicing Out of Bounds

- Python does not allow out of bounds indexing for individual characters of strings
 - Does allow out of bounds indices for slices
- Given: `str1 = "Python"`
 - Then `print(str1[7])`  `print(str1[-7])` 
 - These are OK

```
str1[-10:10] is "Python"
```

```
str1[-10:3] is "Pyt"
```

```
str1[2:10] is "thon".
```

Type Converter Functions

- So far, we have encountered 3 types of values in our programs: *integers*, *strings* and *floats*.
- The corresponding data types are: **int**, **str** and **float** respectively.
- The type converter functions: `int()`, `str()` and `float()` convert their arguments to type `int`, `str` and `float` respectively.
- Note: The function name is the same as the corresponding data type. But the function takes an argument, enclosed in parenthesis.

Type Converter Functions

- The type converter functions: `int()`, `str()` and `float()` convert their arguments to type `int`, `str` and `float` respectively.
 - `int(5.2)`
 - `5`
 - `str(5.2)`
 - `'5.2'`
 - `int('7')`
 - `7`
 - `float(7)`
 - `7.0`
 - `float('ab')`
 - `ValueError`

Converter to int

- The `int()` function takes float or str value and converts it to int.
 - For floats, it discards the decimal portion i.e. *truncates towards zero*.

```
https://NavyblueVirtualArraylist.ajaekel.repl.run
Python 3.6.1 (default, Dec 2015, 13:05:11)
[GCC 4.8.2] on linux
> int(5.7)
5
> int(5.99999)
5
> int(-4.258)
-4
> int(0.0)
0
> int('123')
123
> int('12.3')
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ValueError: invalid literal for int() with base 10: '12.3'
> int(15)
15
> int('abc')
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ValueError: invalid literal for int() with base 10: 'abc'
> 
```


Converter to float

- The float() function takes int or str value and converts it to int.
 - str must be syntactically valid

```
Python 3.6.1 (default, Dec 2015, 13:05:11)
[GCC 4.8.2] on linux
> float(254)
254.0
> float(4.56)
4.56
> float(0)
0.0
> float('45')
45.0
> float('27.8')
27.8
> float('-4')
-4.0
> float('2+4')
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ValueError: could not convert string to float: '2+4'
> float('abc')
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ValueError: could not convert string to float: 'abc'
> 
```

Converter to str

- The str() turns its argument into a str.

```
Python 3.6.1 (default, Dec 2015, 13:05:11)
[GCC 4.8.2] on linux
> str(54)
'54'
> str(-4.8)
'-4.8'
> str(ju78*/)
File "<stdin>", line 1
    str(ju78*/)
        ^
SyntaxError: invalid syntax
> str('abc')
'abc'
> str(x)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
NameError: name 'x' is not defined
> str(2+3)
'5'
```

eval() Function

- `eval(expression)`: The *expression* argument is evaluated as a Python expression.
- *expression* is a **str** argument
 - `eval('5.2')`
 - 5.2
 - `eval('3.4*6.2')`
 - 21.08
 - `eval(7+1)`
 - Error
 - `eval('7+1')`
 - 8

End

Section 2

Chapter 2

An Introduction to Programming Using Python
David Schneider

© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.